2025

DEVOIR MAISON PROJET

MME ROUBEAU

CA TANAVONG

SOMMAIRE

- Page de garde 1
- Sommaire 2
- Objectif atteint 3
- Outils utilisés 4
- Algorithme 5-9
- Programme Java 10-16
- Difficultés rencontrés 17
- Conclusion 17

OBJECTIF DU PROJET :

L'objectif de ce projet était de travailler sur 3 modules.

Le premier projet avait pour but de faire recommencer l'utilisateur autant de fois qu'il le souhaite, il devait seulement saisir des villes existantes parmi 10 villes.

Le deuxième projet avait pour but de faire la méthode de tri pour trier un vecteur Tab de N.

Le dernier projet avait pour but d'utiliser la méthode de tri par propagation et de faire un jeu d'essai.
.

OUTILS UTILISÉS

Pour ce projet, j'ai utilisé Eclipse, ChatGPT et du Java.

Eclipse , c'st un environnement de développement (IDE) qui sert à écrire, tester et exécuter du code en Java.

ChatGPT , c'est une intelligence artificielle conçue pour comprendre et générer du texte.

Java , c'est un langage de programmation utilisé pour créer des applications sur PC, mobile et web.

ALGORITHME DE CHAQUES MODULES

Module 1 :

```
Déclaration des variables :
  villes : tableau de chaînes de caractères
  distances : tableau à 2 dimensions d'entiers
  villeDepart, villeArrivee : chaînes de caractères
  indexDepart, indexArrivee, distance : entiers
  continuer : booléen

Initialiser villes avec les noms des villes
Initialiser distances avec les distances entre les villes

Tant que continuer est vrai :
  Demander "Entrez la ville de départ : "
  Lire villeDepart
  indexDepart = chercherIndex(villes, villeDepart)
  Si indexDepart est égal à -1 :
      Afficher "❌ Ville non trouvée. Veuillez entrer une ville
valide."
    Passer à la prochaine itération (continuer la boucle)
```

```
  Demander "Entrez la ville d'arrivée : "
  Lire villeArrivee
  indexArrivee = chercherIndex(villes, villeArrivee)
  Si indexArrivee est égal à -1 :
    Afficher "❌Ville non trouvée. Veuillez entrer une ville
valide."
    Passer à la prochaine itération (continuer la boucle)

  distance = distances[indexDepart][indexArrivee]
  Afficher "("📌 Distance entre " + villeDepart + " et " +
villeArrivee + " : " + distance + " kms"

  Demander "Voulez-vous faire une autre recherche ?
(oui/non) : "
  Lire réponse
  Si réponse n'est pas égal à "oui" (insensible à la casse) :
    continuer = faux

Afficher " Programme terminé. Merci !"

Fonction chercherIndex(villes, ville) :
  Pour chaque index i de 0 à longueur de villes :
    Si villes[i] est égal à ville (insensible à la casse) :
      Retourner i
  Retourner -1
```

Module 2 :

```
Algorithme "Tri_Par_Selection"

Variables :
   Numérique : N, i, j, minIndex, temp
   Tableau Numérique : tab

Début
    Répéter
       Afficher "Entrez le nombre d'éléments (N > 5) :"
       Saisir N
   Jusqu'à ce que N > 5

    Pour i allant de 0 à N - 1 Faire
       Afficher "Entrez l'élément ", i + 1, " :"
       Saisir tab[i]
   FinPour

 Afficher "Déroulement du tri par sélection :"
   Pour i allant de 0 à N - 2 Faire
       minIndex ← i
       Pour j allant de i + 1 à N - 1 Faire
           Si tab[j] < tab[minIndex] Alors
               minIndex ← j
           FinSi
       FinPour
```

```
temp ← tab[i]
      tab[i] ← tab[minIndex]
      tab[minIndex] ← temp

       Afficher "Étape ", i + 1, " : "
      Pour j allant de 0 à N - 1 Faire
         Afficher tab[j], " "
      FinPour
      Afficher ""
   FinPour

    Afficher "Tableau trié :"
   Pour i allant de 0 à N - 1 Faire
      Afficher tab[i], " "
   FinPour

Fin
```

Module 3 :

```
Algorithme " Tri par la méthode de la propagation"
Variable indicée : Tab[N] de type numérique
Variables : Numérique = N,I,Ech,J,Sauve,Fin

Début
(*Saisie de la taille du tableau*)
Afficher " Quelle est la taille de votre tableau ?"
Saisir N
```

```
(*Remplissage du tableau à trier*)
Pour I allant de 1 à N
Afficher " Veuillez saisir le contenu de la case de rang
",I
Saisir Tab[I]
FPour

(*Sauvegarde de la valeur de N pour la réutiliser après
le tri *)
Sauve <- N

(*Traitement*)
TantQue N -> 0
  I <- 1

  Fin <- 0
  TantQue I < N
   Si Tab[I]> Tab[I+1] Alors Ech <- Tab[I]
          Tab[I] <- Tab [I +1]
          Tab [I+1] <- Ech
   Fsi
   I <- I + 1
  FtantQue
  N<- N-1
 FinTantQue

(*Affichage du tableau trié*)
Pour I allant de 1 à Sauve
  Afficher Tab[I]
 FPour
Fin
```

CODE JAVA DE CHAQUES MODULES

Module 1 :

```
public class Exo1 {
   public static void main(String[] args) {
      // Déclaration des variables
      String[] villes;
      int[][] distances;
      String villeDepart, villeArrivee;
      int indexDepart, indexArrivee, distance;
      boolean continuer = true;

      // Initialisation des villes (triées alphabétiquement)
      villes = new String[]{"Brest", "Grenoble", "Lille", "Lyon",
"Marseille", "Nantes", "Paris", "Rennes", "Strasbourg",
"Toulouse"};

      // Initialisation de la matrice des distances (correspondant
aux villes)
      distances = new int[][]{
         {0, 996, 723, 890, 1286, 305, 564, 245, 1026, 884},
         {996, 0, 750, 104, 286, 711, 576, 747, 505, 543},
         {723, 750, 0, 668, 979, 593, 224, 515, 524, 905},
         {890, 104, 668, 0, 316, 607, 472, 645, 434, 467},
         {1286, 286, 979, 316, 0, 890, 769, 938, 750, 400},
         {305, 711, 593, 607, 890, 0, 386, 106, 832, 559},
         {564, 576, 224, 472, 769, 386, 0, 348, 447, 681},
         {245, 747, 515, 645, 938, 106, 348, 0, 799, 665},
         {1026, 505, 524, 434, 750, 832, 447, 799, 0, 901},
         {884, 543, 905, 467, 400, 559, 681, 665, 901, 0}
      };
```

```
 // Boucle principale pour permettre plusieurs recherches
      while (continuer) {
          // Demande la ville de départ
          villeDepart = Saisie.lire_String("Entrez la ville de
départ : ");

          // Vérifie si la ville existe
          indexDepart = chercherIndex(villes, villeDepart);
          if (indexDepart == -1) {
              System.out.println("❌ Ville non trouvée. Veuillez
entrer une ville valide.");
              continue; // Recommence la boucle
          }

          // Demande la ville d'arrivée
          villeArrivee = Saisie.lire_String("Entrez la ville
d'arrivée : ");

          // Vérifie si la ville existe
          indexArrivee = chercherIndex(villes, villeArrivee);
          if (indexArrivee == -1) {
              System.out.println("❌ Ville non trouvée. Veuillez
entrer une ville valide.");
              continue; // Recommence la boucle
          }

          // Récupère la distance et l'affiche
          distance = distances[indexDepart][indexArrivee];
          System.out.println("📌 Distance entre " + villeDepart
+ " et " + villeArrivee + " : " + distance + " kms");
```

```
    // Récupère la distance et l'affiche
            distance = distances[indexDepart][indexArrivee];
            System.out.println("📌 Distance entre " + villeDepart
+ " et " + villeArrivee + " : " + distance + " kms");

            // Demande si l'utilisateur veut recommencer
            String reponse = Saisie.lire_String("Voulez-vous
faire une autre recherche ? (oui/non) : ");
            if (!reponse.equalsIgnoreCase("oui")) {  // On arrête
si l'utilisateur ne tape pas "oui" (insensible à la casse)
                continuer = false;
            }
        }

        System.out.println("🚀 Programme terminé. Merci !");
    }

    // Méthode pour chercher l'index d'une ville dans le
tableau
    public static int chercherIndex(String[] villes, String ville) {
        for (int i = 0; i < villes.length; i++) {
            if (villes[i].equalsIgnoreCase(ville)) {  // Comparaison
insensible à la casse
                return i; // Retourne l'index si trouvé
            }
        }
        return -1; // Retourne -1 si la ville n'existe pas
    }
}
```

Module 2 :

```
package Exercices;

public class Exo2 {
   public static void main(String[] args) {

 // Déclaration des variables
   int N, i, j, minIndex, temp;

   // 1. Demander le nombre d'éléments (N doit être > 5)
   do {
      System.out.print("Entrez le nombre d'éléments (N > 5) : ");
      N = Saisie.lire_int(null); // Utilisation de la méthode fournie
pour la saisie
   } while (N <= 4);

   // 2. Initialiser le tableau et demander les valeurs à l'utilisateur
   int[] tab = new int[N];
   for (i = 0; i < N; i++) {
      System.out.print("Entrez l'élément " + (i + 1) + " : ");
      tab[i] = Saisie.lire_int(null); // Utilisation de la méthode
fournie
   }

   // 3. Tri par sélection avec affichage progressif
   System.out.println("\nDéroulement du tri par sélection :");
   for (i = 0; i < N - 1; i++) {
      minIndex = i;
      for (j = i + 1; j < N; j++) {
         if (tab[j] < tab[minIndex]) {
            minIndex = j;
```

```
    }
        }
        // Échanger tab[i] avec le plus petit élément trouvé
        temp = tab[i];
        tab[i] = tab[minIndex];
        tab[minIndex] = temp;

        // Affichage de l'état du tableau après chaque
itération
        System.out.print("Étape " + (i + 1) + " : ");
        for (int k = 0; k < N; k++) {
            System.out.print(tab[k] + " ");
        }
        System.out.println();
    }

    // 4. Afficher le tableau trié final
    System.out.println("\nTableau trié :");
    for (i = 0; i < N; i++) {
        System.out.print(tab[i] + " ");
    }
}
}
```

Module 3 :

```
package Exercices;

public class Exo3 {

 public static void main(String[] args) {
  // Déclaration des variables
        int N, i, ech, sauve;

        // 1. Saisie de la taille du tableau
        System.out.print("Quelle est la taille de
votre tableau ? ");
        N = Saisie.lire_int(null);

        // Déclaration et remplissage du tableau
        int[] tab = new int[N];
        for (i = 0; i < N; i++) {
          System.out.print("Veuillez saisir le
contenu de la case de rang " + (i + 1) + " : ");
            tab[i] = Saisie.lire_int(null);
        }
```

```
// Sauvegarde de la valeur de N
sauve = N;
// 2. Tri par propagation (tri à bulles)
while (N > 0) {
    i = 0;
    while (i < N - 1) {
    if (tab[i] > tab[i + 1]) {
       ech = tab[i];
       tab[i] = tab[i + 1];
       tab[i + 1] = ech;
    }
    i++;
   }
   N--;
}
// 3. Affichage du tableau trié
System.out.println("\nTableau trié :");
for (i = 0; i < sauve; i++) {
System.out.print(tab[i] + " ");
      }
    }
}
```

DIFFICULTÉS RENCONTRÉES

Lors de ce projet , j'ai eu quelques difficultés, je n'avais pas compris sur mes codes java , sur l'exercice 2 et le 3. J'avais un problème au "Saisie.lire_int", je ne comprenais pas et quand j'ai ajouté entre paramètre "null", le code fonctionnait.

CONCLUSION

Ce projet m'a permis de mettre en pratique des concepts en algorithmique, le tri de tableaux.

En commençant à développer un programme qui est capable de calculer la distance entre les villes.

Ces 3 modules m'ont permis d'apprendre sur le tri , tri a bulle , le tri par sélection.

Pour ces projets , j'ai utilisés l'intelligence artificielle et l'aide des programmes qu'on avait pratiqués en cours.